

UNIT 2 – PART 1: STACKS & QUEUES

Stacks: Abstract Data Type, Primitive Stack operations: Push & Pop, Array and Linked Implementation of Stack in C, Application of stack: Prefix and Postfix Expressions, Evaluation of postfix expression, Iteration and Recursion- Principles of recursion, Tail recursion, Removal of recursion Problem solving using iteration and recursion with examples such as binary search, Fibonacci numbers, and Hanoi towers. Tradeoffs between iteration and recursion.

Queues: Operations on Queue: Create, Add, Delete, Full and Empty, Circular queues, Array and linked implementation of queues in C, Dequeue and Priority Queue.

◇ 1. What is a Stack?

► Definition:

A **Stack** is a **linear data structure** that follows the rule:

👉 **LIFO (Last In, First Out)**

Means — the element inserted last is removed first.

► Real-life Example:

- **Stack of plates:**
- The last plate placed on top is the first one taken out.
- **Undo operation** in MS Word or VS Code — uses a stack internally.

◇ 2. Abstract Data Type (ADT) of Stack

A **Stack ADT** defines **what operations can be done**, not **how** they are implemented.

► Stack ADT Operations:

1. **Push(x)** → Insert an element x on the top
2. **Pop()** → Remove the top element
3. **Peek() / Top()** → See the top element without removing it
4. **isEmpty()** → Check if stack is empty
5. **isFull()** → Check if stack is full (in array implementation)

◇ 3. Primitive Stack Operations

► (a) Push Operation (Insertion)

- Adds a new element at the top of the stack
- Increase the top index by 1

```
void push(int stack[], int *top, int value, int size) {  
    if (*top == size - 1)  
        printf("Stack Overflow\n");  
    else {  
        *top = *top + 1;  
        stack[*top] = value;  
    }  
}
```

► (b) Pop Operation (Deletion)

- Removes the topmost element
- Decrease the top index by 1

```
void pop(int stack[], int *top) {  
    if (*top == -1)  
        printf("Stack Underflow\n");  
    else {  
        printf("Popped: %d\n", stack[*top]);  
        *top = *top - 1;  
    }  
}
```

◇ 4. Implementation of Stack

► (a) Array Implementation

Stack is represented using a fixed-size array.

Operation	Description	Time
Push	Add element at top	O(1)
Pop	Remove element from top	O(1)

Limitations:

- Fixed size → can cause **overflow** if stack is full.

► (b) Linked List Implementation

Each element (node) contains:

- **Data**
- **Pointer to next node**

Top pointer always points to the **top node**.

Structure Example:

```
struct Node {  
    int data;  
    struct Node* next;  
};
```

Push using Linked List:

```
void push(struct Node** top, int value) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct  
Node));  
    newNode->data = value;  
    newNode->next = *top;  
    *top = newNode;  
}
```

Pop using Linked List:

```
void pop(struct Node** top) {  
    if (*top == NULL)  
        printf("Stack Underflow\n");  
    else {  
        struct Node* temp = *top;  
        *top = (*top)->next;  
        free(temp);  
    }  
}
```

✓ **Advantage:** No overflow (size grows dynamically)

◇ 5. Applications of Stack

Application	Description
Expression Evaluation	Used in converting and evaluating Prefix/Postfix expressions
Function Calls	System uses stack to store return addresses
Undo Operation	Used in text editors
Parentheses Checking	Used to check valid brackets $\{ [()] \}$
Recursion Handling	Each recursive call is stored in stack

◇ 6. Prefix and Postfix Expressions

Type	Example (for $A + B * C$)
Infix	$A + B * C$
Prefix (Polish)	$+ A * B C$
Postfix (Reverse Polish)	$A B C * +$

Why use Postfix?

- No need for brackets
- Easy to evaluate using stack

◇ 7. Evaluation of Postfix Expression

Algorithm (Step-by-Step):

1. Scan the expression left to right.
2. If operand $\rightarrow$ push to stack.
3. If operator $\rightarrow$ pop two operands, perform operation, push result.
4. Final result $\rightarrow$ top of stack.

Example:

Expression: $2\ 3\ 1\ *\ +\ 9\ -$

Step	Action	Stack
Read 2	Push	[2]
Read 3	Push	[2,3]
Read 1	Push	[2,3,1]
Read *	Pop 1,3 $\rightarrow 3 \times 1 = 3$	[2,3] $\rightarrow$ [2,3]
Read +	Pop 3,2 $\rightarrow 2 + 3 = 5$	[5]
Read 9	Push	[5,9]
Read -	Pop 9,5 $\rightarrow 5 - 9 = -4$	[-4]

✓ Final Result = -4

◇ 8. Recursion and Iteration

➤ (a) Iteration

A process that repeats steps using **loops (for, while)**.

Example:

```
for (int i = 1; i <= 5; i++)  
    printf("%d ", i);
```

➤ (b) Recursion

A **function calling itself**.

Example:

```
int factorial(int n) {  
    if (n == 1)  
        return 1;  
    else  
        return n * factorial(n - 1);  
}
```

◇ 9. Principles of Recursion

1. Must have a **base condition** (stopping point).
2. Each recursive call should **move toward base case**.
3. The system uses a **stack** to store each call.

◇ 10. Tail Recursion

A **tail recursive function** is when **the last statement** in the function is the recursive call.

Example:

```
void display(int n) {
    if (n == 0) return;
    printf("%d ", n);
    display(n - 1); // last statement → Tail recursion
}
```

◇ 11. Removal of Recursion

Recursion can be converted into **iteration** using a **stack**.

Example (Recursive to Iterative):

```
// Recursive
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

// Iterative
int factorialIter(int n) {
    int fact = 1;
    for (int i = 1; i <= n; i++)
        fact *= i;
    return fact;
}
```

◇ 12. Problem Solving using Iteration and Recursion

► (a) Binary Search

Recursive:

```
int binarySearch(int arr[], int low, int high, int key) {
    if (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] > key)
            return binarySearch(arr, low, mid - 1, key);
        else
            return binarySearch(arr, mid + 1, high, key);
    }
    return -1;
}
```

► (b) Fibonacci Numbers

Recursive:

```
int fib(int n) {  
    if (n <= 1) return n;  
    return fib(n - 1) + fib(n - 2);  
}
```

Iterative:

```
void fib_iter(int n) {  
    int a = 0, b = 1, c;  
    for (int i = 0; i < n; i++) {  
        printf("%d ", a);  
        c = a + b;  
        a = b;  
        b = c;  
    }  
}
```

► (c) Tower of Hanoi Problem

Recursive Solution:

```
void hanoi(int n, char from, char to, char aux) {  
    if (n == 1)  
        printf("Move disk 1 from %c to %c\n", from, to);  
    else {  
        hanoi(n - 1, from, aux, to);  
        printf("Move disk %d from %c to %c\n", n, from, to);  
        hanoi(n - 1, aux, to, from);  
    }  
}
```

◇ 13. Trade-offs between Iteration and Recursion

Feature	Iteration	Recursion
Memory Use	Less	More (uses stack)
Speed	Usually faster	Slightly slower
Code Simplicity	Long code	Short, cleaner
Termination	Loop condition	Base condition
Best For	Simple repetitive tasks	Divide & conquer problems (like QuickSort, Hanoi)

Topic: Queues

◇ What is a Queue?

A **Queue** is a **linear data structure** that follows the **FIFO** principle —

➡ **First In, First Out**

This means the **element inserted first** is **removed first** — just like people standing in a line (queue) at a ticket counter.

Example (Real-life analogy)

- Imagine a queue at a movie ticket counter:
 - The person who comes **first** buys the ticket **first**.
 - The person who comes **last** will get the ticket **last**.

◇ Basic Queue Operations

Operation	Description	Example
Create	Make an empty queue	Queue of size 5 created
Add (Enqueue)	Insert element at the rear end	Add(10) → [10]
Delete (Dequeue)	Remove element from the front end	Delete() → removes 10
Full	When queue has no more space	size = max → queue is full
Empty	When queue has no elements	size = 0 → queue is empty

◇ Representation of Queue

A queue can be represented in **two main ways**:

1. Array Implementation
2. Linked List Implementation

1. Queue using Array

◇ Structure

We use:

- An array `queue[MAX]`
- Two integer variables:
 - `front` → points to the first element
 - `rear` → points to the last element

◇ Initialization

```
#define MAX 5
int queue[MAX];
int front = -1;
int rear = -1;
```

◇ Enqueue Operation (Add)

```
void enqueue(int item) {
    if (rear == MAX - 1)
        printf("Queue is FULL\n");
    else {
        if (front == -1)
            front = 0;
        rear++;
        queue[rear] = item;
        printf("%d inserted\n", item);
    }
}
```

Example:

```
enqueue(10) → Queue = [10]
enqueue(20) → Queue = [10, 20]
```

◊ Dequeue Operation (Delete)

```
void dequeue() {  
    if (front == -1 || front > rear)  
        printf("Queue is EMPTY\n");  
    else {  
        printf("%d deleted\n", queue[front]);  
        front++;  
    }  
}
```

Example:

dequeue() → removes 10 → Queue = [20]

◊ Check Full / Empty

```
if (rear == MAX - 1) → Queue is FULL  
if (front == -1 || front > rear) → Queue is EMPTY
```

🌀 2. Circular Queue

◊ Problem with Normal Queue

In array implementation, when elements are deleted from the front, **those positions are wasted** even if space exists at the start.

◊ Solution — Circular Queue

In a **Circular Queue**, the last position is connected back to the first position.
So, if the queue is not full, **rear** can wrap around to the start.

◊ Formula for Circular Movement

```
rear = (rear + 1) % MAX  
front = (front + 1) % MAX
```

◇ Example

MAX = 5

Insert 10, 20, 30, 40, 50 → Queue full

Dequeue 10, 20 → front moves forward

Insert 60, 70 → goes to start positions again (circular behavior)

◇ Conditions

Full: (front == 0 && rear == MAX-1) || (front == rear + 1)

Empty: front == -1

3. Queue using Linked List

◇ Structure

Each node contains:

- data
- next pointer

```
struct Node {  
    int data;  
    struct Node *next;  
};  
struct Node *front = NULL, *rear = NULL;
```

◇ Enqueue (Insert at rear)

```
void enqueue(int item) {  
    struct Node *newNode = malloc(sizeof(struct Node));  
    newNode->data = item;  
    newNode->next = NULL;  
    if (rear == NULL)  
        front = rear = newNode;  
    else {  
        rear->next = newNode;  
        rear = newNode;  
    }  
}
```

◊ Dequeue (Delete from front)

```
void dequeue() {
    if (front == NULL)
        printf("Queue is EMPTY\n");
    else {
        struct Node *temp = front;
        printf("%d deleted\n", temp->data);
        front = front->next;
        if (front == NULL)
            rear = NULL;
        free(temp);
    }
}
```

✅ **Advantage:** No fixed size → queue grows dynamically.

↔ 4. Deque (Double Ended Queue)

A **Deque** allows **insertion and deletion from both ends**.

◊ Types:

1. **Input Restricted Deque** → insertion only at one end, deletion both ends
2. **Output Restricted Deque** → deletion only at one end, insertion both ends

◊ Example

```
Front: [10, 20, 30] :Rear
insertRear(40) → [10,20,30,40]
deleteFront() → [20,30,40]
insertFront(50) → [50,20,30,40]
```

🏆 5. Priority Queue

In a **Priority Queue**, each element is associated with a **priority**.

The element with **higher priority** is **served first**, not necessarily the one inserted first.

◇ Example

Element	Priority
Task A	2
Task B	1
Task C	3

Dequeue order → Task C → Task A → Task B
(because $3 > 2 > 1$)

◇ Implementation

Can be done using:

- **Array** (sorted or unsorted)
- **Linked list**
- **Heap** (for better performance)

UNIT 2 – Most Important Questions & Answers (Stacks + Queues)

🔗 SECTION 1: STACKS

Q1. Define Stack. Explain its operations with example.

► **Definition:**

A **stack** is a **linear data structure** that follows the **LIFO (Last In First Out)** principle. The element inserted last is removed first.

► **Operations on Stack:**

1. **Push** → Insert an element on top.
2. **Pop** → Remove element from top.
3. **Peek/Top** → See top element.
4. **isEmpty()** → Check if stack is empty.
5. **isFull()** → Check if stack is full.

► **Example (Step by Step):**

Stack size = 5
Push(10) → [10]
Push(20) → [10, 20]
Pop() → removes 20 → [10]

► **Diagram:**

Top → 20
10

✓ **LIFO:** Last in (20), first out (20)

Q2. Write a C program for array implementation of stack.

```
#include <stdio.h>
#define SIZE 5

int stack[SIZE];
int top = -1;

void push(int value) {
    if (top == SIZE - 1)
        printf("Stack Overflow\n");
    else
        stack[++top] = value;
}

void pop() {
    if (top == -1)
        printf("Stack Underflow\n");
    else
        printf("Popped: %d\n", stack[top--]);
}

void display() {
    for (int i = top; i >= 0; i--)
        printf("%d ", stack[i]);
    printf("\n");
}
```

```
int main() {
    push(10);
    push(20);
    pop();
    display();
    return 0;
}
```

✅ **Output:**

Popped: 20

10

Q3. What are the applications of stack?

Application	Explanation
Expression Evaluation	Used to evaluate postfix and prefix expressions
Expression Conversion	Convert infix → postfix
Function Calls	Each function call is pushed on stack
Undo Operation	Used in editors
Parenthesis Matching	Check balanced brackets {[()]}
Recursion	Each recursive call stored in system stack

Q4. What is postfix expression? Explain postfix evaluation with example.

► **Definition:**

A **postfix (Reverse Polish)** expression is one in which the **operator comes after operands**.

Example:

Infix: A + B * C

Postfix: A B C * +

► **Evaluation Algorithm:**

1. Scan expression left to right.
2. If operand → push into stack.
3. If operator → pop 2 operands, apply operator, push result.
4. Final result → top of stack.

► **Example:** Expression: 2 3 1 * + 9 -

Step	Stack
Push 2	[2]
Push 3	[2,3]
Push 1	[2,3,1]
* → (3×1=3)	[2,3] → [2,3]
+ → (2+3=5)	[5]
Push 9	[5,9]
- → (5-9=-4)	[-4]

✓ **Result = -4**

Q5. Differentiate between iteration and recursion.

Feature	Iteration	Recursion
Definition	Looping using for or while	Function calling itself
Uses	Repetition tasks	Divide and conquer
Memory	Less	More (stack used)
Speed	Faster	Slower
Termination	Loop condition	Base case
Example	for(i=1;i<=n;i++)	fact(n) = n * fact(n-1)

Q6. Write recursive and iterative code for Fibonacci series.

► **Recursive:**

```
int fib(int n) {
    if (n <= 1)
        return n;
    else
        return fib(n - 1) + fib(n - 2);
}
```

► **Iterative:**

```
void fib_iter(int n) {
    int a = 0, b = 1, c;
    for (int i = 0; i < n; i++) {
        printf("%d ", a);
        c = a + b;
        a = b;
        b = c;
    }
}
```



```
}  
}
```

Q7. Explain Tower of Hanoi problem using recursion.

► Problem:

Move n disks from **Source (A)** to **Destination (C)** using **Auxiliary (B)** rod.

► Rules:

1. Move only one disk at a time.
2. Never place a larger disk on a smaller one.

► Recursive Logic:

```
void hanoi(int n, char A, char B, char C) {  
    if (n == 1)  
        printf("Move disk 1 from %c to %c\n", A, C);  
    else {  
        hanoi(n - 1, A, C, B);  
        printf("Move disk %d from %c to %c\n", n, A, C);  
        hanoi(n - 1, B, A, C);  
    }  
}
```

Q8. Explain trade-offs between iteration and recursion.

Aspect	Iteration	Recursion
Memory	Less	More (stack frames)
Code Size	Long	Short & elegant
Execution	Fast	Slightly slow
Use Case	Repeated tasks	Divide-and-conquer
Conversion	Easy to convert	Needs stack

SECTION 2: QUEUES

Q9. Define Queue. Explain its operations with diagram.

► Definition:

A **queue** is a **linear data structure** that follows the **FIFO (First In First Out)** rule.
The element inserted first is removed first.

► **Diagram:**

Front → [10][20][30] ← Rear

► **Operations:**

1. **Enqueue** → Insert element at rear
2. **Dequeue** → Remove element from front
3. **isFull()** → Queue full
4. **isEmpty()** → Queue empty

Q10. Write array implementation of queue in C.

```
#include <stdio.h>
#define SIZE 5

int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int value) {
    if (rear == SIZE - 1)
        printf("Overflow\n");
    else {
        if (front == -1) front = 0;
        queue[++rear] = value;
    }
}

void dequeue() {
    if (front == -1 || front > rear)
        printf("Underflow\n");
    else
        printf("Deleted: %d\n", queue[front++]);
}

void display() {
    for (int i = front; i <= rear; i++)
        printf("%d ", queue[i]);
}

int main() {
    enqueue(10); enqueue(20);
```

```
    dequeue();  
    display();  
}
```

✅ **Output:**

Deleted: 10
20

Q11. What is a circular queue? Why is it needed?

► **Problem:**

In a normal queue, after a few deletions, free space at the beginning can't be reused → causes **waste of memory**.

► **Solution:**

In **Circular Queue**, last position is **connected back to first**, forming a circle.

Formula:

$\text{rear} = (\text{rear} + 1) \% \text{size}$

✅ **Advantage:** Reuses memory efficiently.

Q12. Differentiate between simple queue and circular queue.

Basis	Simple Queue	Circular Queue
Structure	Linear	Circular
Memory Use	Wasted space	Efficient use
rear increment	rear++	$\text{rear} = (\text{rear} + 1) \% \text{size}$
Overflow	When $\text{rear} = \text{size} - 1$	When next $\text{rear} = \text{front}$
Example Use	Ticket line	CPU scheduling

Q13. What is a Deque? Mention its types.

► **Definition:**

Deque (Double-Ended Queue) allows **insertion and deletion at both ends**.

► **Types:**

1. **Input-Restricted Deque** → Insert at one end, delete at both.
2. **Output-Restricted Deque** → Delete at one end, insert at both.

✅ **Example Use:** Undo/Redo operations in text editors.

Q14. What is a Priority Queue?

► Definition:

In a **Priority Queue**, each element is assigned a **priority value**.
The element with **highest priority** is processed first.

► Example:

Job	Priority
A	2
B	5
C	1

Processing order → B → A → C

✓ **Used In:** CPU scheduling, hospital emergency systems.

Q15. Write short notes on:

(a) Queue Overflow

When queue is **full** (no space to insert new element).

Condition:

```
rear == size - 1
```

(b) Queue Underflow

When queue is **empty** (no element to delete).

Condition:

```
front == -1 || front > rear
```